



UDC 004.451:004.272

CLASSICAL INTERPROCESS COMMUNICATION MECHANISMS IN LINUX OS: REVIEW AND LATENCY ANALYSIS FOR HPC SCENARIOS

Danyliuk I.M.*Candidate of Physical and Mathematical Sciences, Associate Professor**ORCID: 0000-0002-6637-9557**Yuriy Fedkovych Chernivtsi National University, Chernivtsi, Kotsiubynskoho 2, 58002*

Abstract. This paper reviews five classical interprocess communication (IPC) mechanisms of the Linux operating system – anonymous pipes, named pipes (FIFOs), message queues (System V and POSIX), shared memory with semaphores, and Unix domain sockets – in the context of high-performance computing (HPC). The architectural features of each mechanism are analysed together with their effect on the key metric – message-passing latency. The methodology of comparative evaluation is described, and measurement results are reported for small messages of 64 to 1024 bytes. The study shows that shared memory with *futex* synchronisation provides the lowest latency (about 0.5 – 1 μ s), pipes and Unix sockets demonstrate moderate latency (about 2 μ s), while message queues exhibit the highest latency (10 – 20 μ s). The paper is the first part of a two-part study; the second part addresses throughput, system-call overhead, and practical recommendations for HPC developers.

Key words: IPC, Linux, HPC, pipe, shared memory, message queue, Unix socket, latency, kernel, *syscall*.

Introduction. Modern HPC tasks – from molecular dynamics simulation to numerical solution of fluid mechanics equations – run mainly on clusters under the Linux operating system. Parallel computing on such systems is organised as a set of interacting processes, and the speed of their data exchange directly defines the final performance of the application. Even MPI (Message Passing Interface), the de-facto standard for distributed computing, falls back on the same IPC primitives provided by the kernel when its processes share a node [1].

Despite the wide use of parallel computing, the question of which IPC mechanism to choose for a given HPC scenario remains open. Existing studies focus mostly on synthetic micro-benchmarks [2, 3] or compare MPI with OpenMP at the application level [4], and a systematic analysis of low-level OS primitives is often missing. At the same time, the choice of primitive defines how much overhead each exchange operation introduces between processes on the same compute node.

The aim of this paper is to provide a detailed review of the architectural features of the classical Linux IPC mechanisms and to analyse their latency for small-message



exchange. Special attention is paid to how the internal structure of each mechanism shapes its performance profile.

Overview of IPC mechanisms in Linux.

Anonymous and named pipes (Pipes and FIFOs). A pipe is the oldest and simplest IPC mechanism in Unix-like systems. It implements a one-way byte stream between two processes through a circular buffer in kernel memory. The default buffer size in Linux is 65 536 bytes (64 KB); the exchange uses standard `read()` and `write()` calls. Anonymous pipes connect only related processes, while named pipes – FIFOs – are accessible to any two processes through the file system [5].

From the kernel viewpoint, a pipe is a special file object, but all transfer happens in memory. This makes pipes fast for small messages. Every exchange needs at least two system calls and two user/kernel transitions, which adds unavoidable overhead.

Message queues. Linux supports two queue standards – System V IPC and POSIX. System V queues are linked lists of structures and let the receiver pick messages not only in FIFO order but also by type. POSIX queues, supported since kernel 2.6.6, are stored in `/dev/mqueue` and offer message prioritisation, asynchronous notification through signals, and a unified file interface [6].

Both types have system limits: the maximum message size (8192 bytes by default) and the maximum number of messages. These can be tuned through `/proc/sys/fs/mqueue/`, but require elevated privileges. Each send or receive passes through the kernel and triggers a context switch.

Shared memory with semaphores. Shared memory is the fastest IPC mechanism: after initial setup, processes read and write data directly in a shared address-space region without kernel involvement. Linux provides two APIs: System V (`shmget`, `shmat`, `shmdt`) and POSIX (`shm_open`, `mmap`). Once memory is mapped, access needs no system call [5].

The main difficulty is that shared memory itself provides no synchronisation. Processes must coordinate through POSIX semaphores (`sem_post`, `sem_wait`) or `futexes`. An uncontended `futex` costs tens of nanoseconds, while a contended call needs the kernel. This asymmetry is critical for HPC: with frequent synchronisations, the



benefit of avoiding system calls is reduced.

Unix domain sockets. Unix domain sockets (UDS) let two processes exchange data through an API similar to Berkeley Sockets, but entirely inside the kernel and without the network stack. They support stream (SOCK_STREAM) and datagram (SOCK_DGRAM) modes. Unlike TCP sockets, which traverse the full network stack (checksum, fragmentation, TCP buffers), Unix sockets are much more efficient [8].

Their advantage is flexibility: native bidirectional communication, available between any two processes on a node without kinship. This makes UDS a convenient choice for client-server architectures inside an HPC node. Still, every send/recv requires a system call.

Methodology of performance evaluation.

To compare IPC mechanisms correctly, representative metrics must be chosen. In the HPC context the most important are: average latency (μs) – the time from writing a message to receiving it in a producer-consumer scheme; tail latency (P99/P99.9) – the 99-th or 99.9-th percentile, important for systems with deterministic response time; system-call overhead – the number of user/kernel transitions per exchange operation.

The work of Immich et al. [2] established a framework for comparing IPC based on a producer-consumer task: one process writes a fixed-size message, the other reads it and returns acknowledgement. This approach isolates the overhead of the mechanism itself from the application logic.

Independent benchmarks such as ipc-bench [7] implement a similar scheme and provide reproducible results for various message sizes. For accurate measurements one must pin processes to CPU cores, disable speculative execution where it affects the result, and warm up caches before measurement, since the first access to an address is always slower than the next ones because of cache misses [3].

Latency analysis for small messages (64 - 1024 bytes).

Shared memory – the lowest latency. For small messages, which is the most typical scenario for signalling between threads of an HPC application, shared memory with futex synchronisation shows the lowest latency – about 0.5 - 1 μs without contention. The reason is that, once memory mapping is set up, every access happens



without entering the kernel.

A benchmark of one million 64-byte messages reports an average response time of about 850 ns for shared memory, while message queues show about 12 μ s at P99 [7]. The gap reflects a basic architectural difference: shared memory transfers data in a single step, while message queues always pass through the kernel.

Pipes and Unix sockets – acceptable latency. Pipes and Unix sockets show similar average latency – about 2 μ s per message for small sizes. This is confirmed by ipc-bench results on a single 3.3 GHz Intel core: UDS – 2 μ s, pipe – 2 μ s, TCP – 6 μ s [7]. Beyond the pipe buffer threshold (64 KB) latency grows because of partial buffer flushing and a repeated system call.

The tail latency (P99) for pipes and sockets is higher than for shared memory: under CPU contention, the kernel scheduler may delay a process waiting in a system call, while a process that "spin-waits" on a variable in shared memory reacts at once.

Message queues – the highest latency. System V and POSIX message queues show the highest average latency – 10 - 20 μ s for small messages. Every send/receive operation passes through the kernel, and the System V linked-list and POSIX priority-tree implementations add their own algorithmic cost.

Yet message queues offer a unique feature: the kernel automatically blocks the consumer when the queue is empty and resumes it on a new message. This avoids busy-waiting and saves CPU time for irregular message flows [6].

Latency summary. Table 1 summarises the latency characteristics of the five reviewed mechanisms for small messages.

Table 1 – Latency comparison of Linux IPC mechanisms for small messages

Mechanism	Avg. latency (64 B)	P99 latency	Syscalls / op	Bidirectional	Process restriction
Pipe (anonymous)	~ 2 μ s	~ 5 - 8 μ s	2	No	Related only
FIFO (named)	~ 2 - 3 μ s	~ 6 - 10 μ s	2	No	Any
System V Msg. Queue	~ 15 - 20 μ s	~ 30 - 40 μ s	2	Yes	Any
POSIX Msg. Queue	~ 12 - 18 μ s	~ 25 - 35 μ s	2	Yes	Any
Shared memory	~ 0.5 - 1 μ s	~ 1.5 - 3 μ s	0*	Yes	Any
Unix domain socket	~ 2 μ s	~ 5 - 8 μ s	2	Yes	Any

* without futex/semaphore synchronisation cost.

Source: data summarised by the authors based on [2, 3, 7].



Impact of kernel security on latency.

After the public disclosure of the Spectre and Meltdown vulnerabilities in 2018, the Linux kernel introduced several protective measures, in particular KPTI (Kernel Page-Table Isolation) and retpoline. These measures significantly increased the cost of system calls. While a simple getpid call previously took about 45 - 90 ns on a modern x86-64 CPU, after the mitigations its cost rose to roughly 100 - 230 ns, depending on the hardware generation and on the support of PCID and other features [4].

For mechanisms that need a system call on every operation (pipes, sockets, message queues), this means an extra 200 - 400 ns per message just for syscall entry and exit. At a million operations per second this cost is about 0.2 - 0.4 ms per core – a noticeable value for HPC applications with a tight exchange loop.

Shared memory is practically unaffected by these changes: the main data path (a regular write and read in address space) does not need any system calls even after the mitigations.

Conclusions.

Five classical IPC mechanisms in Linux were reviewed in terms of their architectural features and latency for small-message exchange.

Shared memory with futex synchronisation shows the lowest latency (0.5 - 1 μ s) because it avoids system calls during the actual data transfer. Anonymous pipes and Unix sockets show acceptable latency (about 2 μ s) and are simple to use. Message queues (System V and POSIX) have the highest latency (10 - 20 μ s), but they offer a convenient abstraction and automatic blocking on an empty queue.

After the introduction of Spectre/Meltdown mitigations, the cost of system calls rose substantially, which further strengthens the advantage of shared memory for HPC scenarios with a tight exchange loop.

The second part of this study addresses throughput comparison for large messages, NUMA effects, and practical recommendations for choosing an IPC mechanism for different HPC scenarios.



References:

1. Adam, J., Besnard, J., Roussel, A., Jaeger, J., Carribault, P., & Pérache, M. (2024). To share or not to share: A case for MPI in shared-memory. In *Recent Advances in the Message Passing Interface. EuroMPI 2024* (pp. 89-102). Springer. https://doi.org/10.1007/978-3-031-73370-3_6
2. Immich, P. K., Bhagavatula, R. S., & Pendse, R. (2003). Performance analysis of five interprocess communication mechanisms across UNIX operating systems. *Journal of Systems and Software*, 68(1), 27-43. [https://doi.org/10.1016/S0164-1212\(02\)00134-6](https://doi.org/10.1016/S0164-1212(02)00134-6)
3. Venkataraman, A., & Jagadeesha, K. K. (n.d.). *Evaluation of inter-process communication mechanisms* [Technical report]. University of Wisconsin-Madison, Department of Computer Sciences. https://pages.cs.wisc.edu/~adityav/Evaluation_of_Inter_Process_Communication_Mechanisms.pdf
4. Orlov, E. (2023). *Measuring context switching and memory overheads for Linux threads: A deep dive into NPTL performance*. <https://eorlov.org/posts/2023/measuring-context-switching-and-memory-overheads-for-linux-threads/>
5. Kerrisk, M. (2010). *The Linux programming interface: A Linux and UNIX system programming handbook*. No Starch Press.
6. Linux man-pages project. (2024). *mq_overview(7) – POSIX message queues overview*. https://man7.org/linux/man-pages/man7/mq_overview.7.html
7. Goldsborough, P. (2022). *ipc-bench: Benchmarks for inter-process-communication techniques* [Software repository]. GitHub. <https://github.com/goldsborough/ipc-bench>
8. Baeldung on Linux. (2025). *IPC performance comparison: Anonymous pipes, named pipes, Unix sockets, and TCP sockets*. <https://www.baeldung.com/linux/ipc-performance-comparison>

Article sent: 08.05.2026

© Danyliuk I.M.